Loop Engineering 实践指南：在 Code Buddy 中构建自主循环系统

原创 腾讯程序员 腾讯技术工程 2026年6月22日 02:22 广东 717人



作者：eliqiao

一、什么是 Loop Engineering

Loop Engineering 是由谷歌工程师 Addy Osmani 提出的 AI 编程新范式。其核心理念是：**围绕大模型构建自主循环运行系统，使 AI 从单次响应工具升级为长期自治代理。**

在传统的 AI 辅助开发中，开发者与 AI 的交互模式是"一问一答"——你发一条指令，AI 回复一次，然后等待下一条指令。这种模式的瓶颈在于：**人成了循环的瓶颈**。每一步都需要人类介入，AI 无法自主推进复杂工作流。

Loop Engineering 的解法是：**让人从循环内部的操作者，转变为循环之上的监督者和目标设定者。** 你定义"做什么"和"何时算完成"，AI 自己决定"怎么做"和"下一步是什么"，直到目标达成或确认不可达。

类比传统工程学中的 PDCA 循环（Plan–Do–Check–Act），在 Loop Engineering 中：

- **模型**是执行者
- **Loop**是控制中枢
- **规则框架**是边界约束

这三者组合，让 AI 在可控范围内自主推进复杂工作流。

为什么现在重要

当模型能力足够强时，**循环设计**成为决定 AI 自主性与可靠性的关键瓶颈。一个设计良好的循环可以让 AI 连续工作数十轮完成复杂重构，而一个设计糟糕的循环可能在第三轮就失控。Loop Engineering 被视为继 Prompt Engineering、Context & Harness Engineering 之后的"AI 编程第三次革命"——开发者角色从"提示词工程师"彻底升级为"AI 系统架构师"。

二、Loop Engineering 与 ReAct 的区别

ReAct（Reasoning + Acting）是目前最主流的 AI Agent 交互范式，由 Yao et al. 在 2022 年提出。其核心模式是：**思考 → 行动 → 观察 → 思考 → 行动 → 观察.....** 交替进行推理和工具调用，直到任务完成。

Loop Engineering 与 ReAct 经常被放在一起讨论，但它们解决的是不同层次的问题。理解两者的区别，是正确实践 Loop Engineering 的前提。

本质关系：Inner Loop vs Outer Loop

ReAct 是 Loop Engineering 的 Inner Loop。



- ReAct 解决的是**单次任务内**"怎么一步步做"的问题
- Loop Engineering 解决的是**跨任务**"做什么、谁来做、何时停、怎么续"的问题

核心差异对比

维度	ReAct	Loop Engineering
关注层次	单次任务的执行过程	跨任务的编排与调度
循环粒度	细粒度（单步工具调用）	粗粒度（整个任务周期）
状态管理	依赖上下文窗口内记忆	状态外置到文件/数据库，每次迭代全新上下文
停止条件	模型自己判断"做完了"	独立评估器验证可度量条件是否满足
验证机制	自我检查（同一模型）	对抗验证（不同模型/独立评估器）
错误恢复	在同一上下文内重试	断点续跑，可跨会话恢复
并行能力	单 Agent 串行	多 Agent 并行 + 工作树隔离
运行周期	单次对话	可持续数小时甚至数天

用一个类比理解

把开发一个软件比作建一栋楼：

- **ReAct** 是工人的工作方式——"我需要砌这面墙，先搬砖，再和水泥，再砌，检查是否平整，不平就修"。它关注的是**单步操作的执行质量**。
- **Loop Engineering** 是项目经理的工作方式——"今天完成地基，明天搭框架，后天装管道，质检通过再进行下一步，不合格就返工"。它关注的是**整体工程进度的编排与质量保障**。

没有工人的砌墙能力（ReAct），楼建不起来；没有项目经理的编排（Loop Engineering），工人可能把墙砌错了地方，或者砌完才发现管道没留口。

ReAct 的局限，Loop Engineering 如何补位

局限 1：上下文窗口有限，长任务必然遗忘

ReAct 在同一个上下文窗口内持续推理。当任务跨越几十步操作时，早期信息被压缩或遗忘，导致 AI "忘记"之前做了什么、为什么这么做。

Loop Engineering 的解法：**状态外置**。每轮迭代从全新上下文开始，从持久化文件中读取状态，不依赖模型记忆。CodeBuddy 中的 Memory、CODEBUDDY.md、Rules 就是这一原则的体现。

局限 2：自我检查存在盲区

ReAct 中同一个模型既写代码又检查代码，容易产生确认偏误——"我写的当然没问题"。

Loop Engineering 的解法：**对抗验证**。执行者和评估者使用不同模型或指令。CodeBuddy 中 `/goal` 的评估器用小模型独立判断，Team 模式中 planner/coder/reviewer 使用不同角色，都是对抗验证的实践。

局限 3：没有跨任务的进度跟踪

ReAct 完成一个任务就结束，没有"做到哪了"的持久记录。如果中途崩溃，只能从头来。

Loop Engineering 的解法：**断点续跑**。通过状态文件记录进度，崩溃后可从断点恢复。CodeBuddy 中 `/goal` 支持 `--resume` 恢复未完成的 goal，Memory 记录跨会话上下文。

局限 4：缺少编排能力

ReAct 是单 Agent 串行执行，无法同时推进多个子任务。

Loop Engineering 的解法：**多 Agent 并行 + 工作树隔离**。CodeBuddy 的 Team 模式支持多个 Agent 同时工作在不同分支上，互不干扰，最后合并结果。

不是替代，是演进

Loop Engineering 不是要替代 ReAct，而是在 ReAct 之上增加编排层。两者关系：

```
Prompt Engineering    → 怎么问（单次交互优化）
  ↓
ReAct                 → 怎么做（单任务内的推理-行动循环）
  ↓
Loop Engineering      → 怎么管（跨任务的编排、验证、状态管理）
```

在 CodeBuddy 中，当你用 `/goal` 设置一个条件时，每一轮内部 AI 仍然使用 ReAct 模式来思考和行动，但 `/goal` 的评估器在 Outer Loop 层面判断整体进度——这就是两层循环的协作。

三、Loop Engineering 的核心架构

五阶段循环机制

Loop Engineering 遵循 **Discover → Plan → Execute → Verify → Iterate** 闭环：

阶段	说明	关键设计
Discover	自动读取 CI 失败、issue、代码审查等信号	输入源要结构化、可订阅
Plan	分解目标为具体步骤	温度适中，避免过早收敛
Execute	执行代码编辑与工具调用	工具调用要幂等、可回滚
Verify	通过测试、lint、类型检查等客观信号验证	验证标准必须客观、可机器判定
Iterate	失败则自动修复并重新循环；成功则进入下一任务	状态要持久化，支持断点续跑

双层循环模型

Loop Engineering 采用双层循环架构：

```
Outer Loop（编排层）
  目标拆解 → 任务分配 → 结果汇总 → 再计划

  Inner Loop（执行层）
    感知 → 推理 → 规划 → 行动 → 观察
      ↑                          ↓
      评估 / 修正 ←──────────────┘
```

- **Inner Loop** = 单个 Agent 的工作循环（等价于 ReAct 模式）
- **Outer Loop** = 编排器管理多个 Inner Loop 的生命周期

六要素构建体系

要素	作用	为什么重要
自动化	提供循环心跳，按计划或事件触发	没有心跳就没有循环
工作树	通过 git 为每个 Agent 创建独立工作目录	并行开发零冲突
技能（SKILL.md）	固化项目知识，避免每次冷启动重新推导	知识复用，降低 token 消耗
连接器（MCP）	打通 issue 系统、CI 等真实工具链	AI 必须能感知和操作真实世界
子智能体	将写代码与检查代码分离，形成对抗验证	避免单一 Agent 自我检查的盲区
状态文件	记录进度，支撑断点续跑	防止上下文遗忘和信息漂移

状态外置哲学

Loop Engineering 的一个关键设计原则：**所有状态存储在外部系统，而非模型的上下文窗口。** 每次循环迭代从一个全新的上下文窗口开始，基于实际持久化内容工作。这彻底解决了模型遗忘、信息漂移与上下文压缩问题。

四、CodeBuddy 中的 Loop Engineering 实现

CodeBuddy 提供了三种核心机制来实现 Loop Engineering，它们分别对应不同的循环驱动模式：

3.1 /goal — 条件驱动的持续工作

`/goal` 是 Loop Engineering 最直接的实现。你设置一个**可验证的完成条件**，CodeBuddy 跨多轮自动工作，直到条件满足。

```
# 基本语法
/goal <完成条件>

# 实际示例
/goal all tests in test/auth pass and the lint step is clean

# 加兜底上限防止无限循环
/goal all tests pass or stop after 20 turns

# 非交互模式（headless 运行）
codebuddy -p "/goal CHANGELOG.md has an entry for every PR merged this week"
```

写好条件的三个关键要素：

要素	说明	示例
可度量的终态	测试结果、构建退出码、文件数等	`all tests in test/auth pass`
可证明方式	明确怎么验证	`` `npm test` exits 0 或 `git status` is clean ``
不可破坏的约束	过程中不能改的东西	`no other test file is modified`

评估机制：每轮结束后，由独立小模型评估器判断条件是否满足，三态结果：

- ✅ `ok: true` — 条件已满足，清除 goal，UI 显示 `✔ Goal achieved`
- 🔄 `ok: false` — 条件未满足，reason 注入 history，继续下一轮
- ❌ `ok: false, impossible: true` — 目标不可达，立即清除 goal

关键设计：**评估器使用小模型**（如 gemini-2.5-flash），快速且便宜，只看 transcript 不调用工具。评估器与执行 Agent 使用不同模型，天然形成对抗验证。

3.2 `/loop` — 时间驱动的循环任务

`/loop` 按时间间隔重复执行指令，适合监控、巡检等持续性场景：

```
# 语法
/loop [时间间隔] <指令>

# 检查 CI/CD 流水线状态
/loop 3m 检查一下流水线是否跑完，把结果告诉我

# 定时运行单元测试
/loop 30m 帮我运行一次单元测试，如果有失败的用例告诉我

# 每小时汇总代码审查待办
/loop 1h 看一下有没有新的 PR 需要我审查
```

特性	说明
最小间隔	1 分钟
每会话上限	50 个任务
自动过期	3 天后自动清除
生命周期	会话级，退出后消失
执行时机	只在会话空闲时触发

3.3 Automations — 跨会话的定时任务

Automations 是持久化的定时任务，不随会话消失，适合需要长期运行的监控场景：

- **Recurring**：按 cron 规则重复执行，如 `FREQ=HOURLY;INTERVAL=1`
- **Once**：指定时间点一次性触发

三种方式的对比

方式	下一轮何时开始	何时停止	适用场景
`/goal`	上一轮结束后立即	评估器确认条件满足	有明确终态的实质性工作
`/loop`	时间间隔触发	主动停或模型判定结束	监控、巡检、定期检查
Automations	按 cron 规则	永久或设有效期	跨会话的长期监控

五、Loop Engineering 架构在 CodeBuddy 中的映射

Loop Engineering 要素	CodeBuddy 实现	说明
自动化（循环心跳）	`/goal`、`/loop`、Automations	三种驱动模式覆盖不同场景
工作树（并行隔离）	Git worktree + Team 模式	多 Agent 并行开发互不干扰
技能（SKILL.md）	Skills 机制	固化项目知识，避免冷启动
连接器（MCP）	MCP 协议	打通 issue、CI、数据库等工具链
子智能体（对抗验证）	Task 工具 + Team 模式	规划者/执行者/评审者三角分工
状态文件	Memory、CODEBUDDY.md、Rules	跨会话知识持久化

六、实践案例

实践 1：使用 /goal 完成模块迁移

场景：将一个旧认证模块从 `auth/legacy` 迁移到新 API，直到所有调用点都能编译且测试通过。

```
/goal all tests in test/auth pass, `npm run build` exits 0, no file in auth/legacy is
```

为什么这样写条件：

1. `all tests in test/auth pass` — 可度量的终态，确保功能不受损

2. `` `npm run build` exits 0 `` — 可证明方式，确保编译通过
3. `no file in auth/legacy is imported anywhere` — 不可破坏的约束，确保旧代码完全移除
4. `or stop after 30 turns` — 兜底上限，防止无限循环

CodeBuddy 的自动循环过程：

1. **Discover**：扫描 `auth/legacy` 目录，识别所有导出和调用点
2. **Plan**：列出需要修改的文件，制定迁移步骤
3. **Execute**：逐个文件修改导入路径，更新调用方式
4. **Verify**：运行 `npm test` 和 `npm run build`，检查编译和测试结果
5. **Iterate**：如果测试失败，分析错误原因，修复后重新运行

每轮结束后，评估器独立判断："所有 auth 测试通过了吗？构建成功了吗？旧文件还有被引用吗？"——只有三个条件全部满足才宣告完成。

实践 2：使用 /loop 实现 CI 监控与自动修复

场景：在提交 PR 后，持续监控 CI 状态，如果失败则尝试自动修复。

```
/loop 2m 检查当前分支的 CI 状态，如果失败了，查看失败日志并尝试修复，修好后提交
```

这个循环的工作流：

```
每 2 分钟触发
  → 读取 CI 状态
  → 如果通过 → 报告成功
  → 如果失败 → 读取错误日志 → 分析原因 → 修改代码 → 提交 → 等待下一轮验证
```

注意：`/loop` 是时间驱动的，不会自动停止。当你确认 CI 通过后，需要手动取消：

```
取消 CI 监控的定时任务
```

实践 3：使用 Team 模式实现对抗验证

场景：复杂重构任务，需要规划者、执行者、评审者三角分工。

```
1. 创建 Team
2. 生成 "planner"：负责分析代码库、制定重构方案
3. 生成 "coder"：根据方案执行代码修改
4. 生成 "reviewer"：独立审查修改结果，确保质量
5. planner 和 reviewer 使用不同模型/指令，形成对抗验证
```

为什么对抗验证重要：单一 Agent 自己检查自己的代码，容易产生盲区——"我写的代码当然没问题"。规划者和评审者使用不同的视角和指令，能发现执行者忽略的问题。

实践 4：使用 Skills 固化项目知识

场景：团队有一套编码规范和架构约定，希望 AI 每次都能遵循。

在 `skills/` 目录下创建 `skill.md`，将项目知识固化：

```
---
name: project-conventions
description: 项目编码规范和架构约定
---

## 架构约定
- 所有 API 调用必须经过 service 层，controller 不直接调用 repository
- 数据库查询必须使用参数绑定，禁止字符串拼接 SQL
- 错误处理遵循统一异常体系...

## 命名规范
- Service 类以 Service 结尾
- Repository 类以 Repository 结尾
...
```

效果：每次 AI 工作时自动加载这些约定，不需要你在每次对话中重复说明。这正是 Loop Engineering 中"技能"要素的体现——**避免每次冷启动重新推导**。

实践 5：使用 MCP 连接器打通工具链

场景：让 AI 能读取 Jira issue、触发 Jenkins 构建、查询数据库。

通过 MCP 协议接入第三方工具：

```
{
  "mcpServers": {
    "jira": {
      "command": "mcp-jira",
      "args": ["--project", "MYPROJ"]
    },
    "jenkins": {
      "command": "mcp-jenkins",
      "args": ["--url", "https://ci.example.com"]
    }
  }
}
```

接入后，在 `/goal` 循环中 AI 可以：

```
/goal all issues labeled "bug" in current sprint have a corresponding test that passes
```

AI 能直接读取 Jira issue 列表、编写测试、运行验证——整个 Discover → Plan → Execute → Verify → Iterate 闭环都在真实工具链中运行。

实践 6：使用 Rules 和 Memory 实现状态外置

场景：长期运行的重构任务，需要跨会话保持上下文。

Rules（`.codebuddy/rules`）：定义硬性约束，AI 每次启动都会读取：

```
- 所有数据库查询必须使用参数绑定
- 修改 API 接口必须同步更新 OpenAPI 文档
- 每个 PR 不超过 300 行变更
```

Memory：记录跨会话的偏好和上下文：

```
- 用户偏好使用 pytest 而非 unittest
- 当前项目使用 Python 3.11，类型注解必须完整
- 用户上次重构到 step 3，下次继续从 step 4 开始
```

效果：每次新的循环迭代从全新的上下文窗口开始时，Rules 和 Memory 提供了"状态外置"——关键信息不依赖模型的记忆，而是从持久化存储中读取。这正是 Loop Engineering "状态外置哲学"的体现。

七、最佳实践与注意事项

写好 /goal 条件的 checklist

1. **终态可度量**：用"测试通过""构建成功""文件数为 0"这类客观指标，避免"代码质量提升"这类主观判断
2. **验证方式明确**：指定用什么命令/工具来验证，如 `` `pytest test/auth` exits 0 ``
3. **约束不可破坏**：明确过程中不能改的东西，如"不修改其他模块的测试"
4. **兜底上限**：始终加 `or stop after N turns`，防止无限循环消耗 token
5. **条件不超过 4000 字符**：这是 `/goal` 的硬性限制

避免常见陷阱

陷阱	说明	解法
验证责任不可转移	无人值守的循环也是无人值守地犯错	关键变更仍需人工审查，Loop 不代替 Code Review
理解债务加速累积	代码库真实状态与开发者理解之间的鸿沟随循环加速扩大	定期 review AI 的变更，保持对代码库的理解
认知投降风险	开发者极易停止独立判断，系统给什么就接受什么	把 AI 当协作者而非权威，质疑每个变更
Token 成本约束	循环运行消耗大量 token	设置 `stop after N turns`，用小模型评估器
条件模糊导致无效循环	条件不够具体，AI 反复尝试但无法满足	条件要写成"AI 自己的输出能证明"的形式

何时用哪种循环模式

```
有明确终态的实质性工作（迁移、重构、实现功能）
  → /goal

持续性监控和巡检（CI 状态、PR 审查、性能回归）
  → /loop

跨会话的长期定期任务（每日构建检查、每周代码审查汇总）
  → Automations

需要多角色协作的复杂任务（规划+执行+评审）
  → Team 模式 + /goal
```

八、总结

Loop Engineering 的核心不是某个具体工具，而是一种**系统设计思维**：把 AI 当作循环中的执行者，你设计循环的规则、验证标准和状态管理。CodeBuddy 提供了完整的工具链来实现这一范式：

- `/goal` 是 Loop 的心跳——条件驱动，自动循环
- `/loop` 是 Loop 的时钟——时间驱动，定期触发
- **Skills** 是 Loop 的知识——固化经验，避免冷启动
- **MCP** 是 Loop 的触手——连接真实工具链
- **Team 模式** 是 Loop 的分身——对抗验证，多角色协作
- **Rules + Memory** 是 Loop 的记忆——状态外置，断点续跑

从"一问一答"到"设定目标、自动循环"，这是 AI 辅助开发从工具到伙伴的质变。而 Loop Engineering 的实践者，正是这一质变的设计师。





阅读 2.3万

留言

写留言

还没有留言，来写首评